

CBS 4GL Development Copilot

Module d'aide au développement 4GL pour CBS / Sopra Banking Amplitude

Objectif : transformer un besoin métier en dossier de développement structuré : analyse, sous-tâches, 4GL, SQL, IHM .per, tests, livraison et rollback.

1. Parcours principal

Besoin métier → Analyse → Domaine CBS → Sous-tâches → Conception fonctionnelle → Conception technique → Génération 4GL / SQL / .per → Contrôles → Tests → Livraison.

2. Saisie du besoin

- Titre et description fonctionnelle
- Domaine bancaire et utilisateurs
- Règles métier
- Données d'entrée et résultat attendu
- Contraintes
- Version d'Amplitude et environnement
- Cas nominaux et erreurs

3. Analyse intelligente

- Résumé et objectif
- Acteurs, préconditions, postconditions
- Règles métier
- Données nécessaires
- Dépendances CBS
- Questions non résolues
- Risques techniques

4. Décomposition automatique en sous-tâches

ID	Sous-tâche	Type	Exemple
ST-01	Analyser le besoin	Fonctionnel	Règles et critères d'acceptation
ST-02	Identifier les données	SQL / CBS	Tables, champs, structures à confirmer
ST-03	Concevoir l'écran	IHM	Champs, boutons, navigation
ST-04	Développer le traitement	4GL	Contrôles et traitement métier
ST-05	Gérer les erreurs	4GL	Messages et codes retour
ST-06	Préparer les tests	Tests	Nominal, erreur, limites
ST-07	Préparer la livraison	RUN/BUILD	Installation, contrôle, rollback

5. Génération du code 4GL

Pour chaque sous-tâche : objectif, type de programme, paramètres, variables, structures, appels, accès aux données, erreurs, transactions et logs.

Structure : déclaration → variables/structures → initialisation → paramètres → contrôles → accès aux données → traitement → erreurs → sortie → journalisation → fin.

```
MAIN
DEFINE p_num_compte CHAR(24)
DEFINE v_solde DECIMAL(18,3)
DEFINE v_code_retour INTEGER
DEFINE v_message CHAR(255)

LET v_code_retour = 0

IF p_num_compte IS NULL OR p_num_compte = "" THEN
LET v_code_retour = 1
LET v_message = "Le numéro de compte est obligatoire"
ELSE
-- Contrôles du format
-- Recherche dans les structures CBS
-- Lecture du solde
-- Gestion des erreurs
-- Préparation de la sortie
END IF
END MAIN
```

Règle : ne jamais inventer une table, un champ, une fonction 4GL, une structure Amplitude ou une API interne. Les éléments inconnus sont marqués « à confirmer ».

6. Génération SQL

Fournir l'objectif, tables, jointures, paramètres, filtres, absence de données, performance, index éventuels et contrôles de sécurité.

```
SELECT compte.numero_compte, compte.statut, compte.solde_disponible
FROM compte
WHERE compte.numero_compte = :p_num_compte;
```

7. Génération IHM .per

- Nom et objectif de l'écran
- Champs de saisie / lecture seule
- Boutons et navigation
- Messages et validations
- Droits d'accès
- Appels vers les traitements 4GL
- Maquette et squelette .per
- Points dépendants de la version Amplitude

8. Mode « Analyse du point de rupture »

Entrées : erreur, log, code 4GL, SQL, capture ou description. Sortie : symptôme → contexte → étape → programme probable → SQL → hypothèses → contrôles → cause racine possible → correction → tests.

Distinction obligatoire : fait observé / hypothèse / information manquante / diagnostic confirmé.

9. Mode « Revue de code 4GL »

- Syntaxe et variables
- Contrôles manquants

- Gestion des erreurs
- SQL et transactions
- Performance
- Lisibilité / conventions
- Compatibilité
- Risques de régression

10. Prompt système recommandé

Tu es CBS 4GL Development Copilot, spécialisé dans les systèmes bancaires CBS, particulièrement Sopra Banking Amplitude.

Mission : transformer un besoin métier en dossier de développement : analyse fonctionnelle, décomposition, conception technique, 4GL, SQL, IHM .per, tests, critères d'acceptation, livraison et rollback.

Règles impératives :

- Ne jamais inventer table, champ, structure, fonction 4GL, API ou programme Amplitude.
- Signaler toute information technique manquante.
- Distinguer faits, hypothèses et éléments à vérifier.
- Adapter les propositions à la version d'Amplitude et à l'environnement fournis.
- Ne jamais présenter le code généré comme directement déployable sans validation.
- Pour la production : procédure, validation, sauvegarde et rollback.
- Tenir compte de la sécurité, des droits, de la traçabilité et de la séparation des environnements.

Format : 1) Résumé 2) Informations manquantes 3) Analyse fonctionnelle 4) Architecture 5) Sous-tâches 6) Conception technique 7) 4GL 8) SQL 9) IHM .per 10) Tests 11) Livraison 12) Synthèse confirmé / proposé / à vérifier / risques / prochaines actions.

11. Modèle de données recommandé

Entité	Principaux champs
DevelopmentRequest	id, title, businessNeed, domain, amplitudeVersion, environment, status
Analysis	requestId, summary, actors, businessRules, assumptions, missingInformation, risks
SubTask	requestId, title, description, type, priority, dependencies, acceptanceCriteria, status
GeneratedArtifact	subTaskId, type, fileName, content, version, validationStatus, reviewComment
TestCase	requestId, title, preconditions, steps, expectedResult, status

12. Types d'artefacts

4GL_PROGRAM · PER_SCREEN · SQL_SCRIPT · PSEUDOCODE · FUNCTIONAL_SPECIFICATION · TECHNICAL_SPECIFICATION · TEST_PLAN · DEPLOYMENT_PLAN · ROLLBACK_PLAN · CODE_REVIEW

13. Interface recommandée

Besoin → saisie du besoin métier

Sous-tâches → plan de développement

Générateur → 4GL / SQL / .per

Tests → validation fonctionnelle

Livraison → dossier technique final

Bouton : « Générer le dossier complet » pour produire spécification fonctionnelle, spécification technique, sous-tâches, 4GL, SQL, .per, tests, critères d'acceptation, procédure de livraison et rollback.

Principe directeur : Ne jamais se précipiter pour conclure. Le Copilot assiste le développeur, explicite ses hypothèses et laisse la validation finale au développeur et aux règles de l'environnement CBS.